PROBLEM SOLVING AND SEARCH

CHAPTER 3

Outline

- Problem-solving agents
- Problem types
- Problem formulation
- Example problems
- Basic search algorithms

Problem-solving agents

A **problem-solving agent- thinking before acting:**

- Knows the world (or assumes it)
- Thinks offline
- Produces a plan
- Executes it

Agent Cycle

1. Agent **perceives** the world
2. Decides **what it wants**
3. Models the problem
4. Simulates outcomes
5. Picks a plan(sequence of actions)
6. Executes step by step

AI never solves the real world — it solves its *model* of the world.

Problem-solving agents

Problem-solving agents work only in static, deterministic, fully observable environments.

```
function Simple-Problem-Solving-Agent( percept) returns an action
    static: seq, an action sequence, initially empty
            state, some description of the current world state
            goal, a goal, initially null
            problem, a problem formulation

    state ← Update-State(state, percept)
    if seq is empty then
        goal ← Formulate-Goal(state)
        problem ← Formulate-Problem(state, goal)
        seq ← Search( problem)
    action ← First(seq); seq ← Rest(seq)
    return action
```

Note: this is offline problem solving; solution executed "eyes closed."
Online problem solving involves acting without complete knowledge.

Problems formulated in terms of atomic states

Example: Romania

Offline Planning

Offline means the agent thinks without interacting with the real environment.

- Think → Act
- Uses world model
- No learning during execution

Because in some environments, mistakes are costly. Planning avoids expensive failures

Online / Learning

Online planning is a decision-making approach where an agent plans and acts at the same time, using feedback from the environment after each action.

- Act → Learn → Act
- Uses feedback
- Adapts during execution

Example: Romania

On holiday in Romania; currently in Arad.
Flight leaves tomorrow from Bucharest

Formulate goal:

be in Bucharest

Formulate problem:

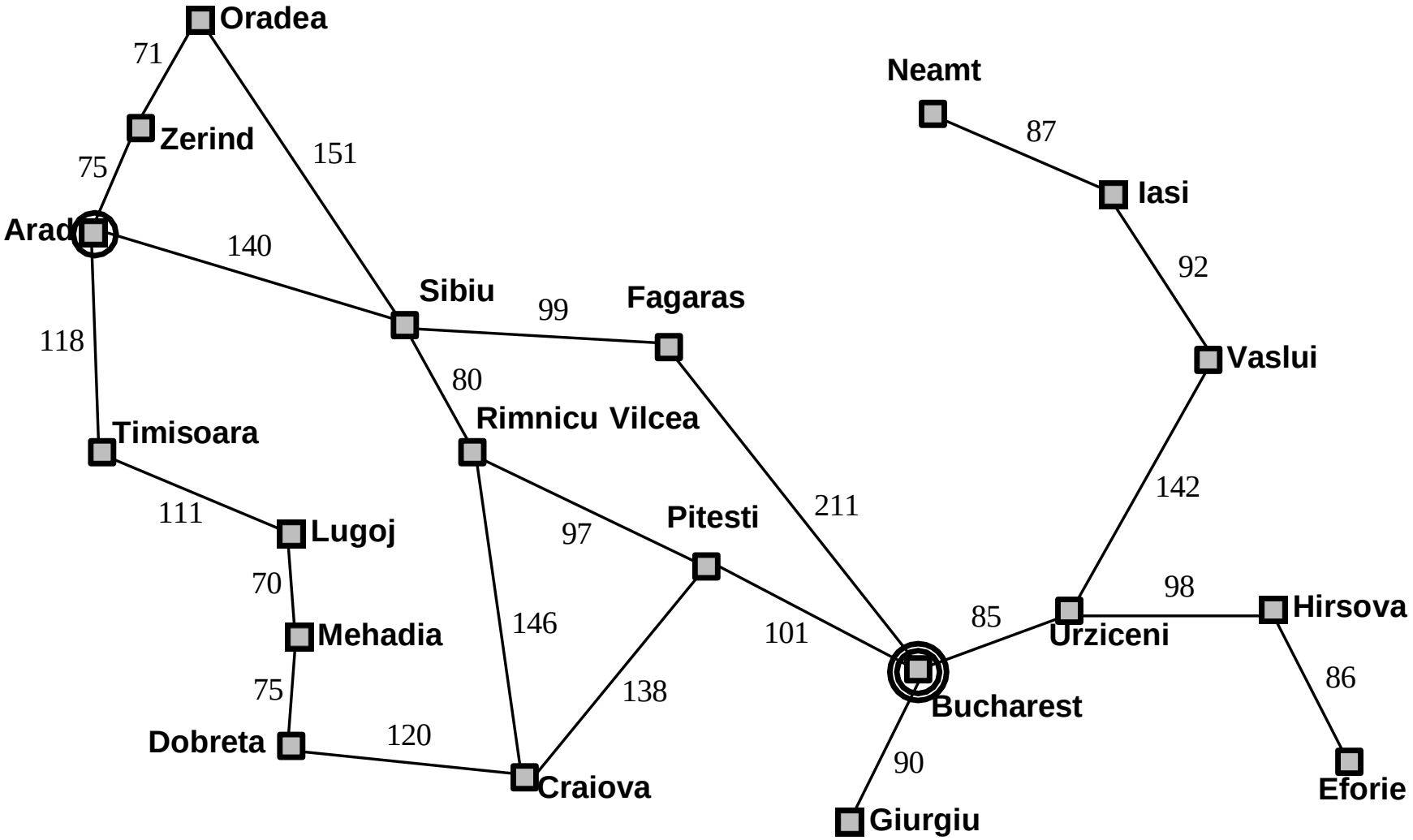
states: various cities

actions: drive between cities

Find solution:

sequence of cities, e.g., Arad, Sibiu, Fagaras, Bucharest

Example: Romania



Problem types

Deterministic, fully observable ==⇒ single-state problem

– Agent knows exactly which state it will be in; solution is a sequence

Non-observable ==⇒ sensorless problem (a.k.a. conformant)

– Agent may have no idea where it is; solution (if any) is a sequence

Nondeterministic and/or partially observable ==⇒ contingency problem

– Percepts provide new information about current state

– Solution is a contingent plan or a policy

– Often interleave search, execution

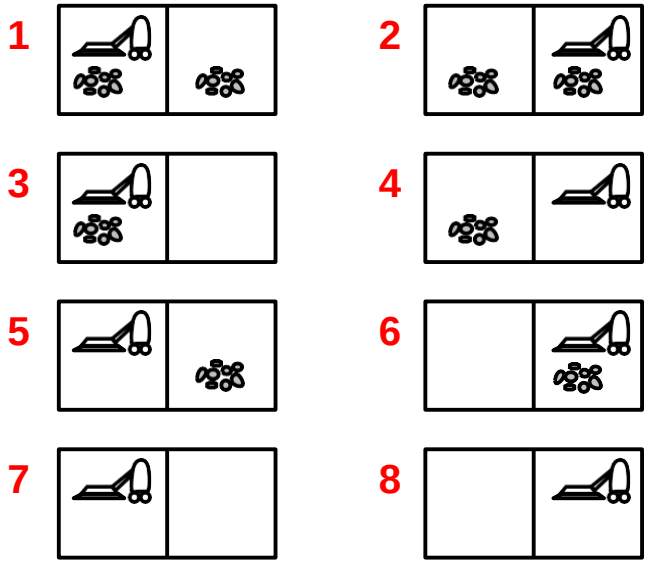
Unknown state space ==⇒ exploration problem ("online")

Learn + act

The agent learns the world by living in it

Example: vacuum world

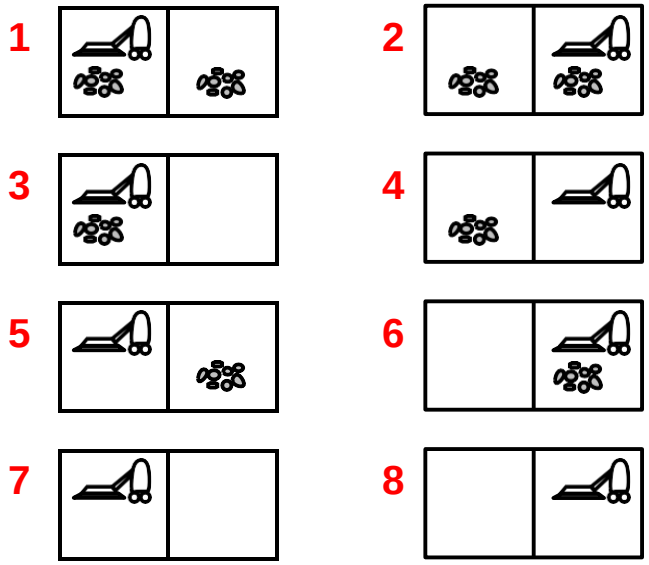
Single-state, start in #5. Solution??



Example: vacuum world

Single-state, start in #5. Solution??
[*Right, Suck*]

Sensorless, start in {1, 2, 3, 4, 5, 6, 7, 8}
e.g., *Right* goes to {2, 4, 6, 8}. Solution??

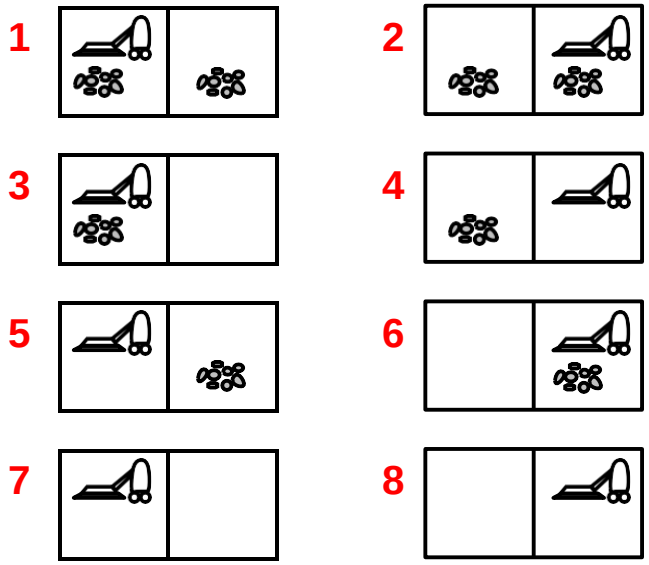


Example: vacuum world

Single-state, start in #5. Solution??
[*Right, Suck*]

Sensorless, start in {1, 2, 3, 4, 5, 6, 7, 8}
e.g., *Right* goes to {2, 4, 6, 8}. Solution??
[*Right, Suck, Left, Suck*]

Contingency, start in #5
Murphy's Law: *Suck* can dirty a clean carpet
Local sensing: dirt, location only.
Solution??

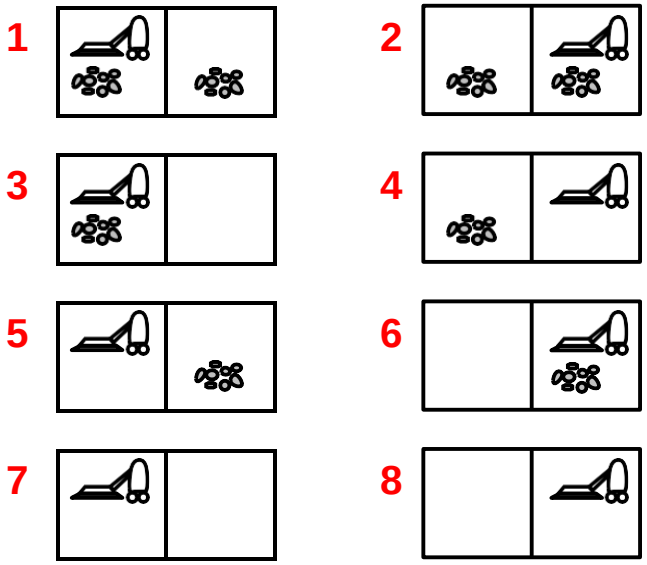


Example: vacuum world

Single-state, start in #5. Solution??
[*Right, Suck*]

Sensorless, start in {1, 2, 3, 4, 5, 6, 7, 8}
e.g., *Right* goes to {2, 4, 6, 8}. Solution??
[*Right, Suck, Left, Suck*]

Contingency, start in #5
Murphy's Law: *Suck* can dirty a clean carpet
Local sensing: dirt, location only.
Solution??
Initial belief state is {5, 7}
[*Right,* if *dirt* then *Suck*]



Sense → Act → Sense → Decide again.

Example: vacuum world

Murphy's Law is used to model nondeterminism in contingency problems, where an action may have undesirable outcomes. The agent therefore plans conditionally and continuously senses to handle worst-case scenarios.

SUMMARY TABLE

Problem Type	**If room is CLEAN**	**If room is DIRTY**
Single-State	Do NOT Suck	Suck
Sensorless	Still Suck	Suck
Contingency	Do NOT Suck	Suck
Unknown	Try & observe	Try & observe

Single-state problem formulation

A problem is defined by four items:

initial state e.g., "at Arad"

successor function $S(x)$ = set of action–state pairs

e.g., $S(Arad) = \{\langle Arad \rightarrow Zerind, Zerind\rangle , . . .\}$

goal test, can be

explicit, e.g., x = "at Bucharest"

implicit, e.g., $NoDirt(x)$

path cost (additive)

e.g., sum of distances, number of actions executed, etc.

$c(x, a, y)$ is the step cost, assumed to be ≥ 0

A solution is a sequence of actions
leading from the initial state to a goal state

Selecting a state space

Real world is absurdly complex

⇒ state space must be abstracted for problem solving

(Abstract) state = set of real states

(Abstract) action = complex combination of real actions

e.g., "Arad → Zerind" represents a complex set of possible routes, detours, rest stops, etc.

For guaranteed realizability, any real state "in Arad" must get to some real state "in Zerind"

(Abstract) solution

= sequence of abstract actions

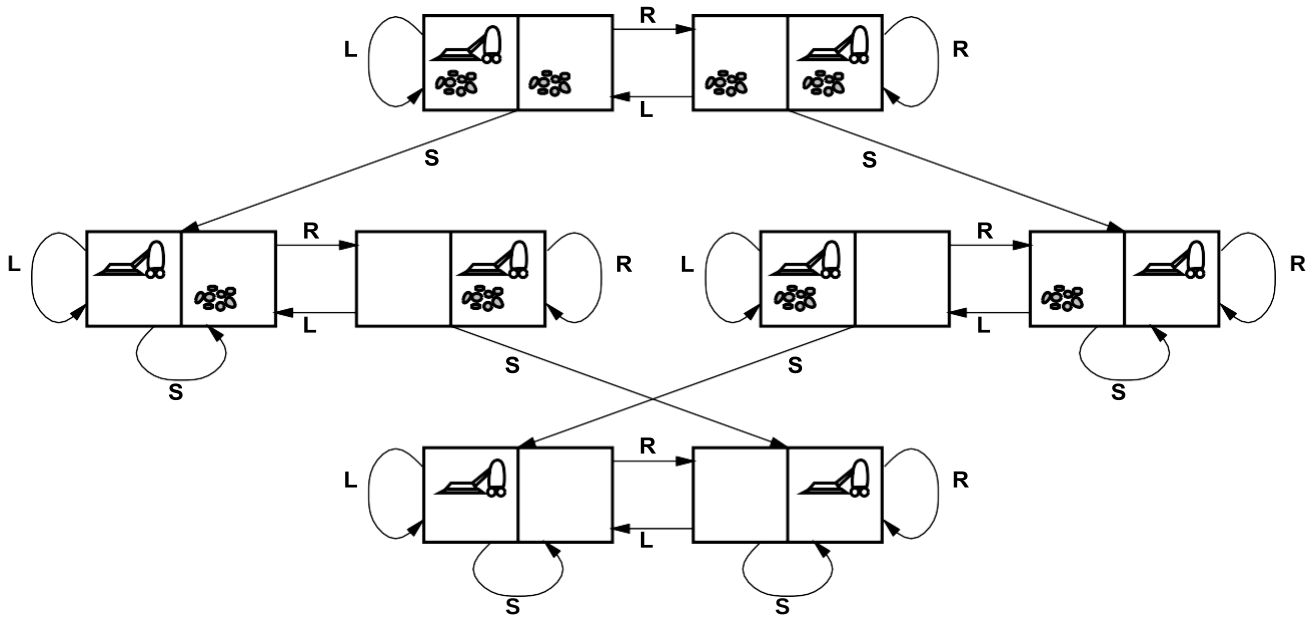
= set of real paths that are solutions in the real world

Each abstract action should be "easier" than the original problem

Example: vacuum world state space graph

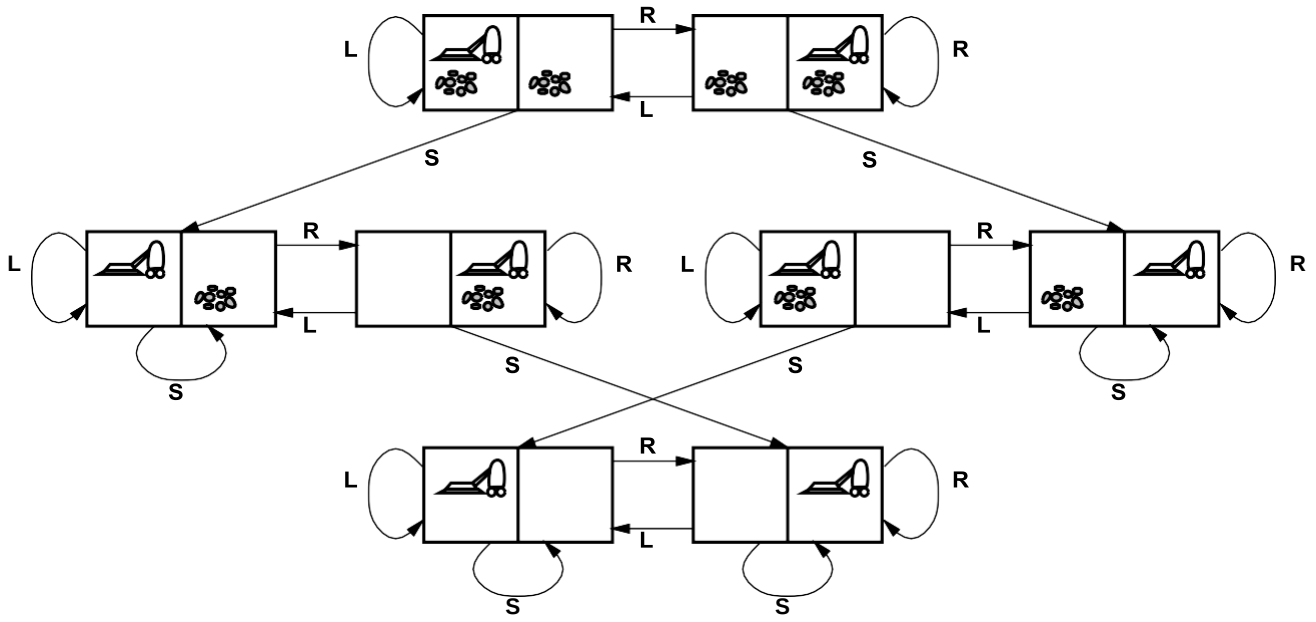
states??
actions??
goal test??
path cost??

Example: vacuum world state space graph



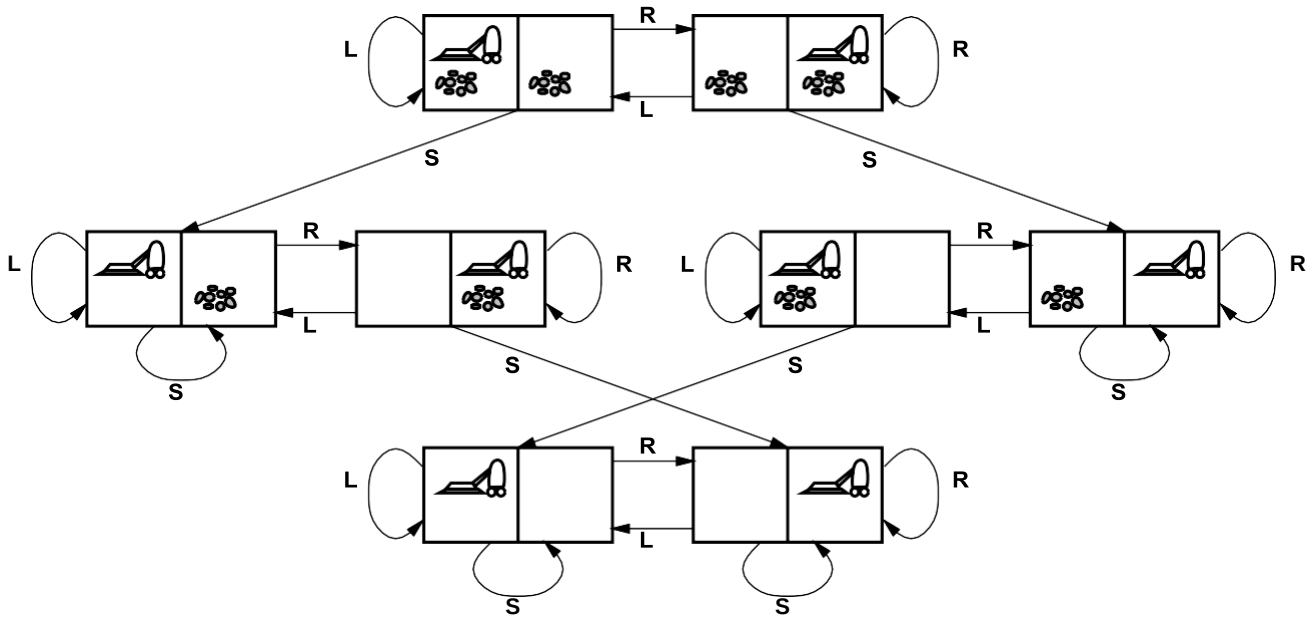
states??: integer dirt and robot locations (ignore dirt amounts etc.)
actions??
goal test??
path cost??

Example: vacuum world state space graph



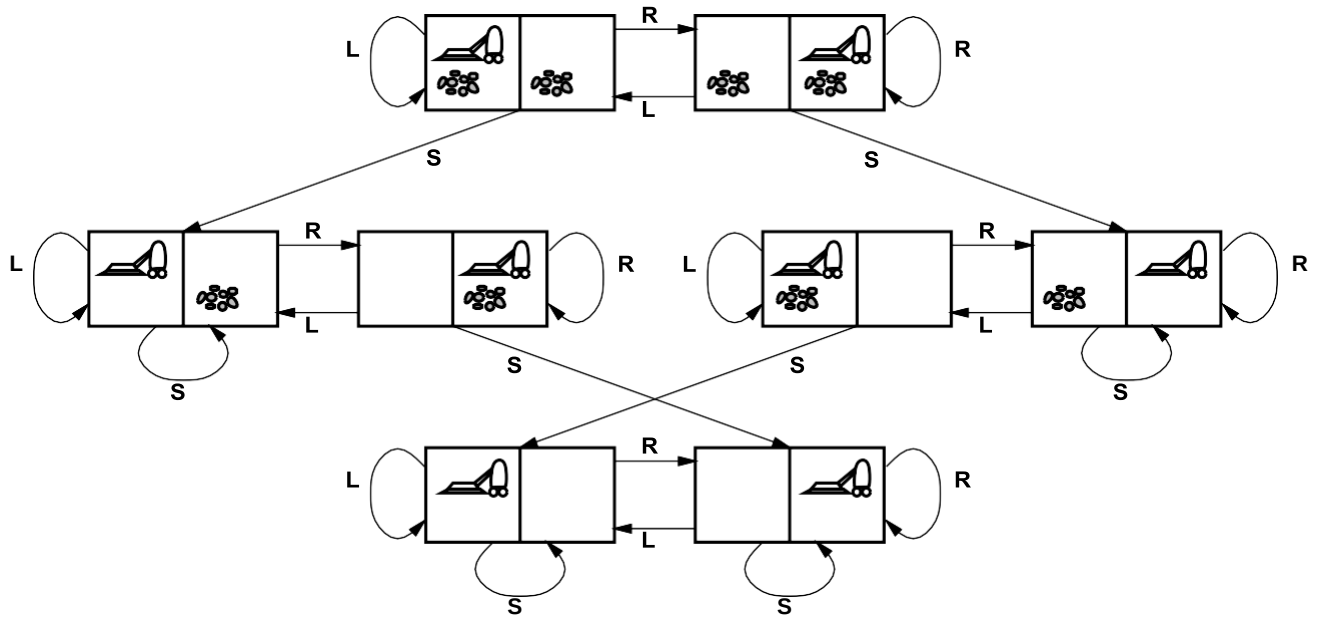
states??: integer dirt and robot locations (ignore dirt amounts etc.)
actions??: *Left*, *Right*, *Suck*, *NoOp*
goal test??
path cost??

Example: vacuum world state space graph



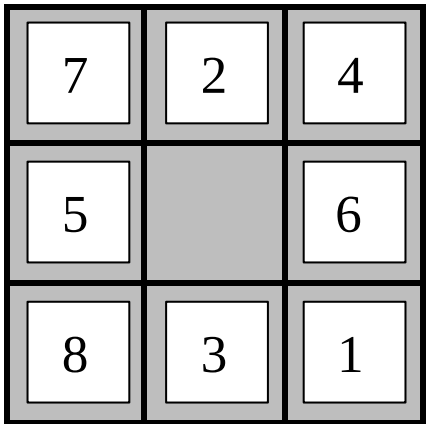
states??: integer dirt and robot locations (ignore dirt amounts etc.)
actions??: *Left*, *Right*, *Suck*, *NoOp*
goal test??: no dirt
path cost??

Example: vacuum world state space graph

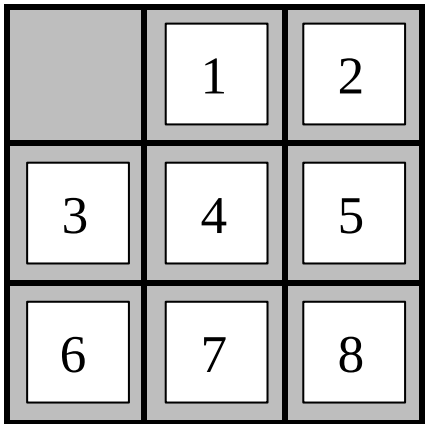


states??: integer dirt and robot locations (ignore dirt amounts etc.)
actions??: *Left*, *Right*, *Suck*, *NoOp*
goal test??: no dirt
path cost??: 1 per action (0 for *NoOp*)

Example: The 8-puzzle



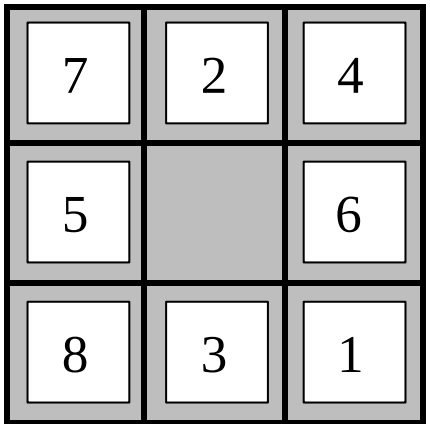
Start State



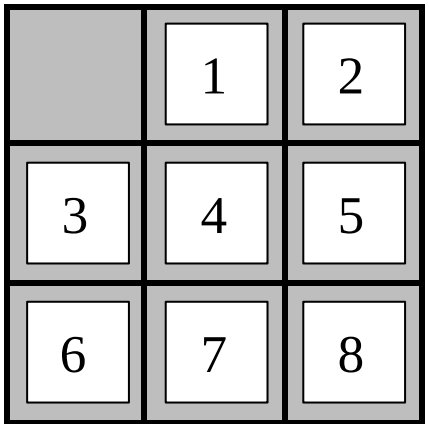
Goal State

states??
actions??
goal test??
path cost??

Example: The 8-puzzle



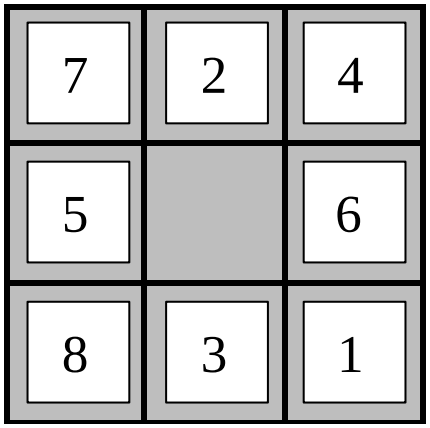
Start State



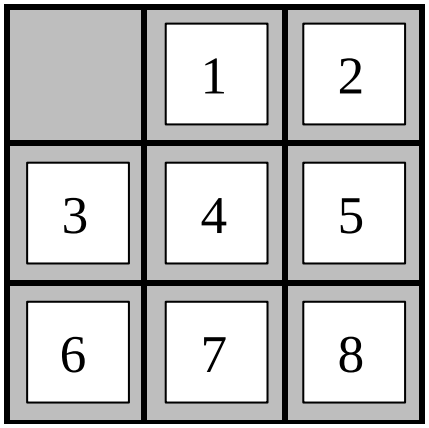
Goal State

states??: integer locations of tiles (ignore intermediate positions)
actions??
goal test??
path cost??

Example: The 8-puzzle



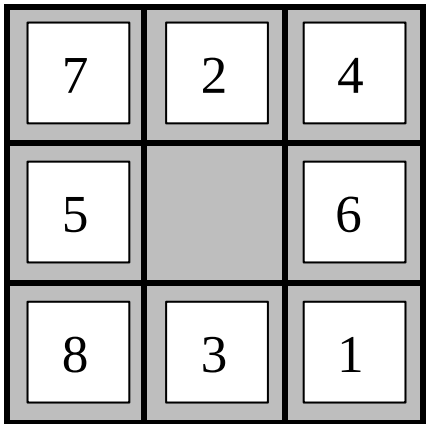
Start State



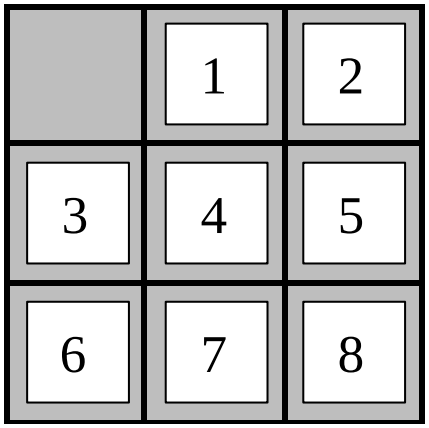
Goal State

states??: integer locations of tiles (ignore intermediate positions)
actions??: move blank left, right, up, down (ignore unjamming etc.)
goal test??
path cost??

Example: The 8-puzzle



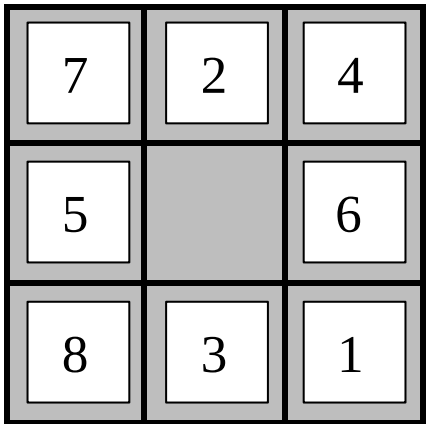
Start State



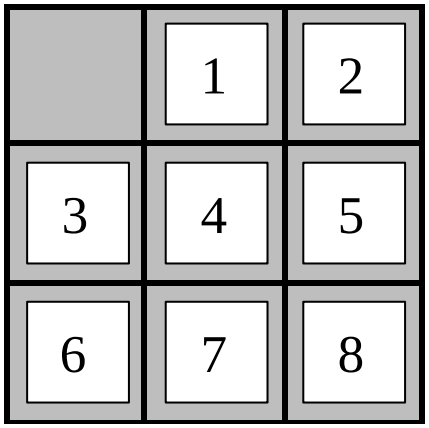
Goal State

states??: integer locations of tiles (ignore intermediate positions)
actions??: move blank left, right, up, down (ignore unjamming etc.)
goal test??: = goal state (given)
path cost??

Example: The 8-puzzle



Start State



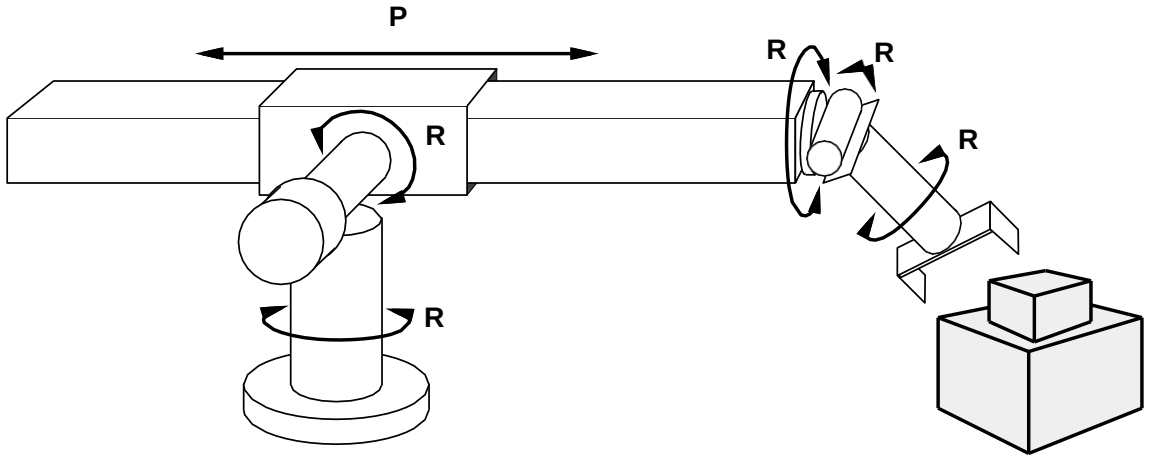
Goal State

states??: integer locations of tiles (ignore intermediate positions)
actions??: move blank left, right, up, down (ignore unjamming etc.)
goal test??: = goal state (given)
path cost??: 1 per move

[Note: optimal solution of *n*-Puzzle family is NP-hard]

Once a problem is properly formulated, AI can solve it.

Example: robotic assembly



states??: real-valued coordinates of robot joint angles
and parts of the object to be assembled

actions??: continuous motions of robot joints

goal test??: complete assembly with no robot included!

path cost??: time to execute

Tree search algorithms

Basic idea:

offline, simulated exploration of state space
by generating successors of already-explored states
(a.k.a. expanding states)
Tree search builds a tree of possibilities.

Root → initial state
Branches → actions
Nodes → new states

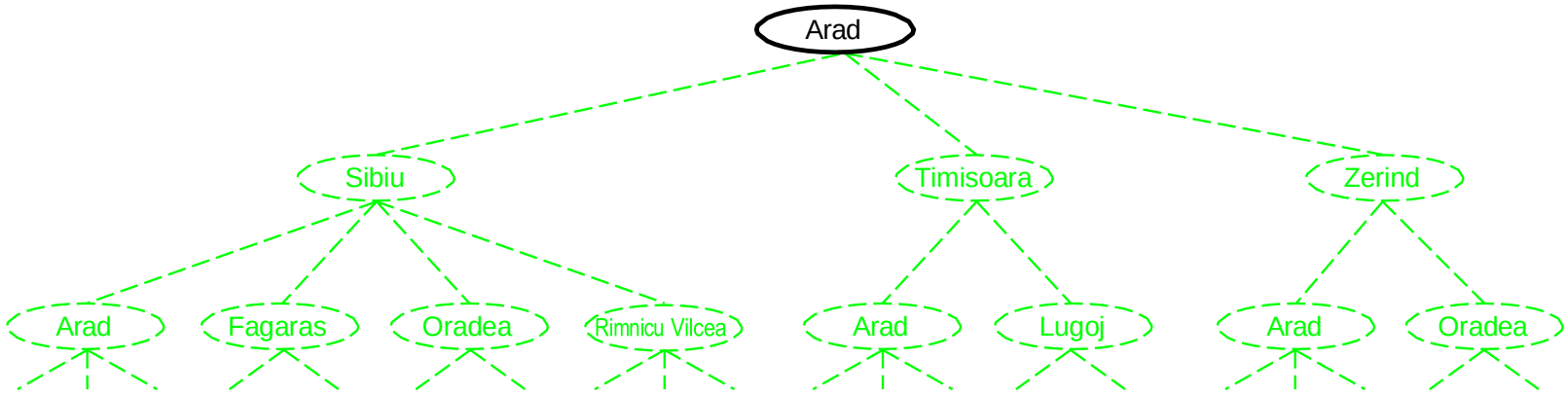
Search is the thinking process of an intelligent agent.

Tree search explores the state space by expanding nodes according to a given strategy until a goal state is found or no nodes remain.

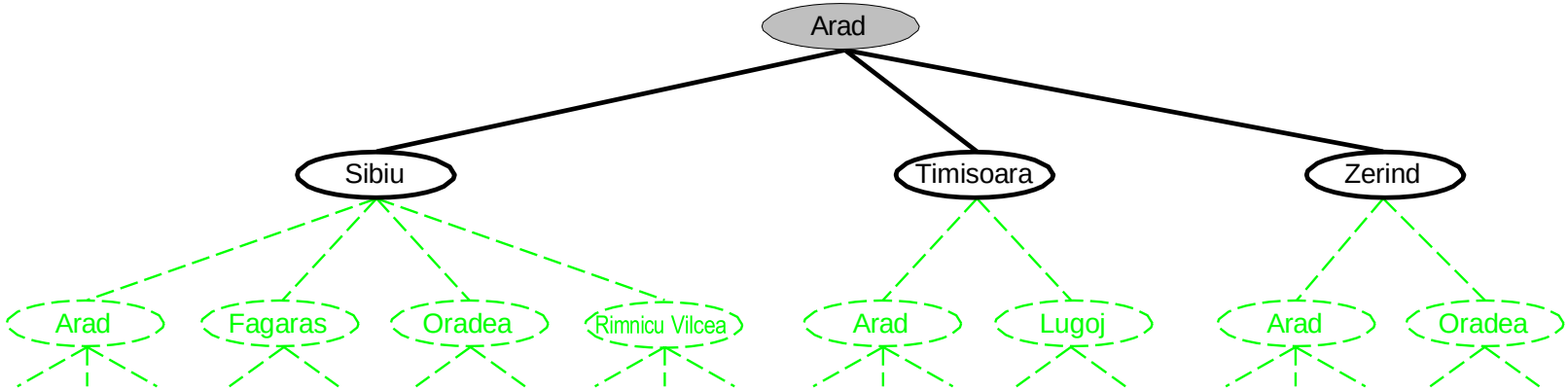
Tree search algorithms

```
function Tree-Search( problem, strategy) returns a solution, or failure initialize
    the search tree using the initial state of problem
    loop do
        if there are no candidates for expansion then return failure choose
        a leaf node for expansion according to strategy
        if the node contains a goal state then return the corresponding solution
        else expand the node and add the resulting nodes to the search tree
    end
```

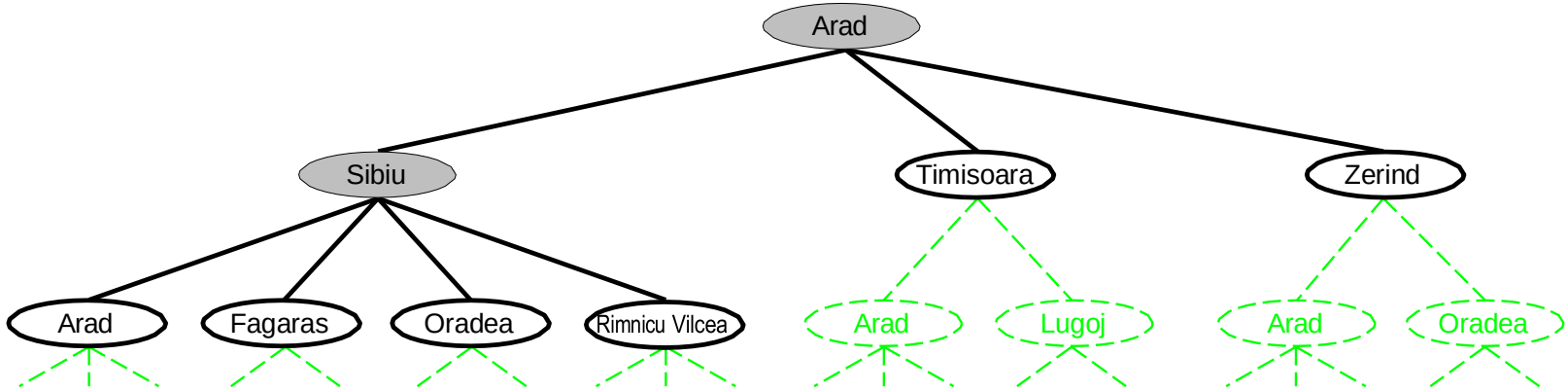
Tree search example



Tree search example

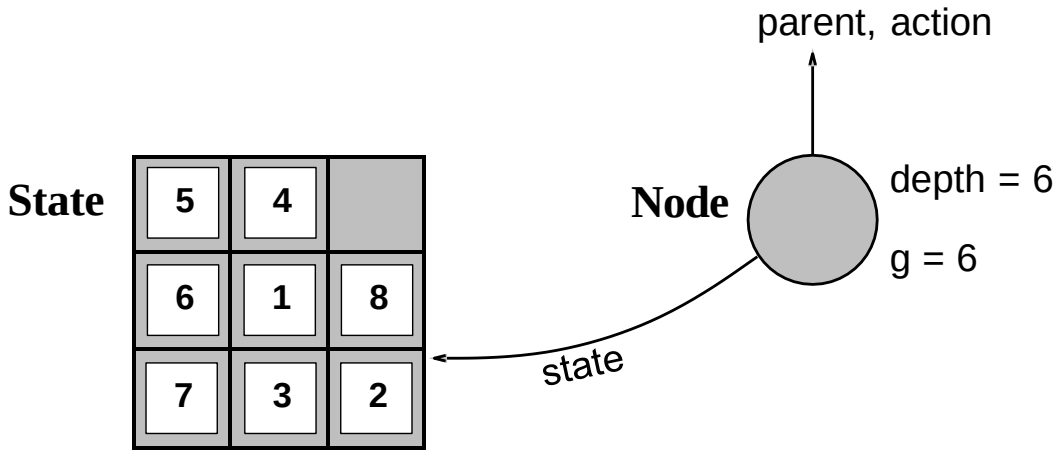


Tree search example



Implementation: states vs. nodes

A state is a (representation of) a physical configuration
A node is a data structure constituting part of a search tree
includes parent, children, depth, path cost $g(x)$
States do not have parents, children, depth, or path cost!
States are facts. Nodes are stories.



The ExPAnd function creates new nodes, filling in the various fields and using the SuccEssoRFn of the problem to create the corresponding states.

Implementation: general tree search

```
function Tree-Search( problem, fringe) returns a solution, or failure
    fringe ← Insert(Make-Node(Initial-State[problem]), fringe)
    loop do
        if fringe is empty then return failure
        node ← Remove-Front(fringe)
        if Goal-Test(problem, State(node)) then return Solution(node)
        fringe ← InsertAll(Expand(node, problem), fringe)
```

```
function Expand( node, problem) returns a set of nodes
    successors ← the empty set; state ← State[node]
    for each action, result in Successor-Fn(problem, state) do
        s ← a new Node
        Parent-Node[s] ← node;  Action[s] ← action;  State[s] ← result
        Path-Cost[s] ← Path-Cost[node]+Step-Cost(state, action, result)
        Depth[s] ← Depth[node] + 1
        add s to successors
    return successors
```

Search strategies

A strategy is defined by picking the order of node expansion

A strategy decides WHICH node to expand NEXT.

Strategies are evaluated along the following dimensions:

- completeness—does it always find a solution if one exists?
- time complexity—number of nodes generated/expanded
- space complexity—maximum number of nodes in memory
- optimality—does it always find a least-cost solution?

Time and space complexity are measured in terms of

- b—maximum branching factor of the search tree
- d—depth of the least-cost solution
- C^*—path cost of the least-cost solution
- m—maximum depth of the state space (may be ∞)

Uninformed search strategies

Uninformed strategies use only the information available in the problem definition

Uninformed Search, also called Blind Search, refers to a class of search algorithms that explore the state space without using any domain-specific knowledge or heuristics.

These algorithms do not know how close a state is to the goal. They only know:

- Initial state
- Available actions
- Goal test
- Path cost (optional)

They systematically explore all possible states until they find a goal.

Uninformed search strategies

Breadth-first search

Uniform-cost search

Depth-first search

Depth-limited search

Iterative deepening search

Breadth-first search

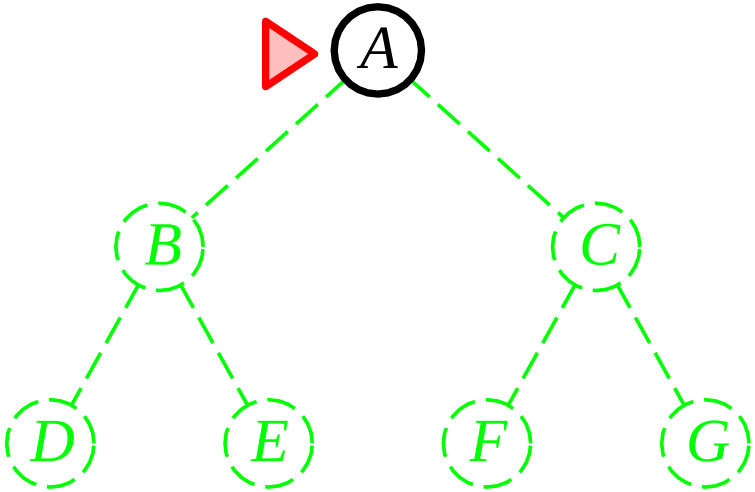
Expand shallowest unexpanded node, Explore **level by level,**

BFS is safe but very expensive in memory

where:
b = branching factor
d = depth of shallowest goal

Implementation:

fringe is a FIFO queue, i.e., new successors go at end

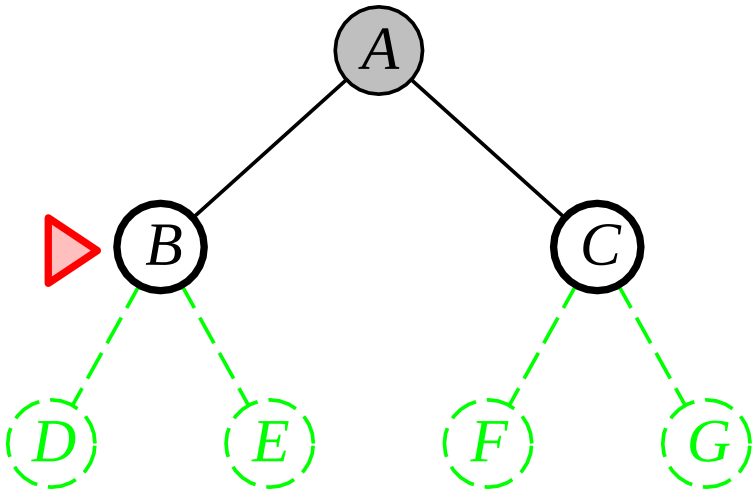


Breadth-first search

Expand shallowest unexpanded node

Implementation:

fringe is a FIFO queue, i.e., new successors go at end

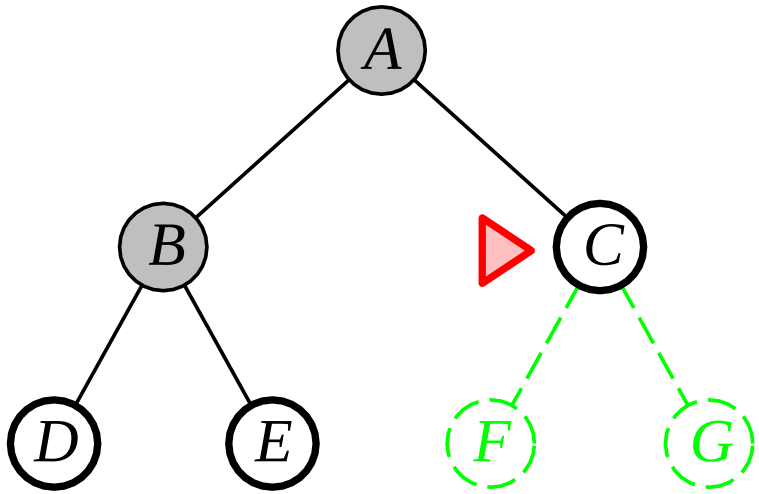


Breadth-first search

Expand shallowest unexpanded node

Implementation:

fringe is a FIFO queue, i.e., new successors go at end

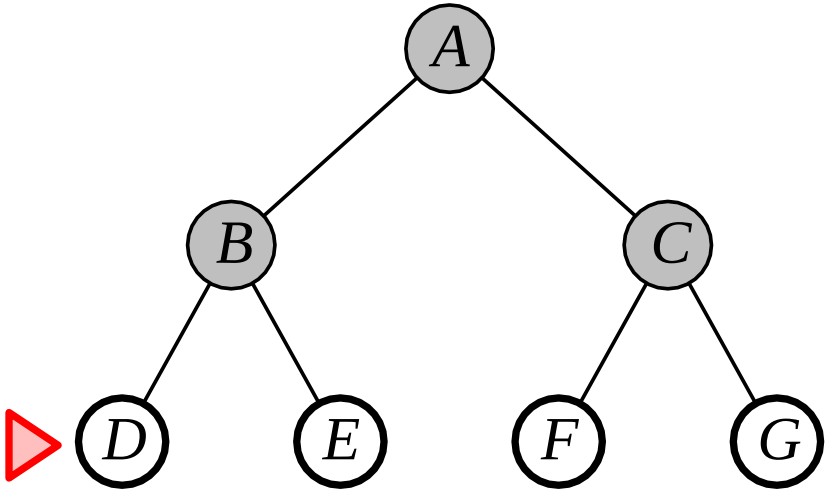


Breadth-first search

Expand shallowest unexpanded node

Implementation:

fringe is a FIFO queue, i.e., new successors go at end



Properties of breadth-first search

Complete??

Properties of breadth-first search

Complete?? Yes (if *b* is finite)

Time??

Properties of breadth-first search

Complete?? Yes (if b is finite)

Time?? $1 + b + b^2 + b^3 + \ldots + b^d + b(b^d - 1) = O(b^{d+1})$, i.e., exp. in d

Space??

Properties of breadth-first search

Complete?? Yes (if b is finite)

Time?? $1 + b + b^2 + b^3 + \ldots + b^d + b(b^d - 1) = O(b^{d+1})$, i.e., exp. in d

Space?? $O(b^{d+1})$ (keeps every node in memory)

Optimal??

Properties of breadth-first search

Complete?? Yes (if b is finite)

Time?? $1 + b + b^2 + b^3 + \ldots + b^d + b(b^d - 1) = O(b^{d+1})$, i.e., exp. in d

Space?? $O(b^{d+1})$ (keeps every node in memory)

Optimal?? No, unless step costs are constant

Space is the big problem; can easily generate nodes at 100MB/sec
so 24hrs = 8640GB.

Time Complexity:
O(b^d)
Space Complexity:
O(b^d)

BFS is fast in finding shallow solutions, but it **dies because of memory explosion**.

Uniform-cost search

Expand least-cost unexpanded node

Implementation:

fringe = queue ordered by path cost, lowest first

Equivalent to breadth-first if step costs all equal where:

C* = cost of optimal solution

ε = minimum edge cost

Complete?? Yes, if step cost $\geq \epsilon$

Time?? # of nodes with $g \leq$ cost of optimal solution, $O(b^{\lceil C^*/\epsilon \rceil})$

where C^* is the cost of the optimal solution

Space?? # of nodes with $g \leq$ cost of optimal solution, $O(b^{\lceil C^*/\epsilon \rceil})$

Optimal?? Yes—nodes expanded in increasing order of $g(n)$

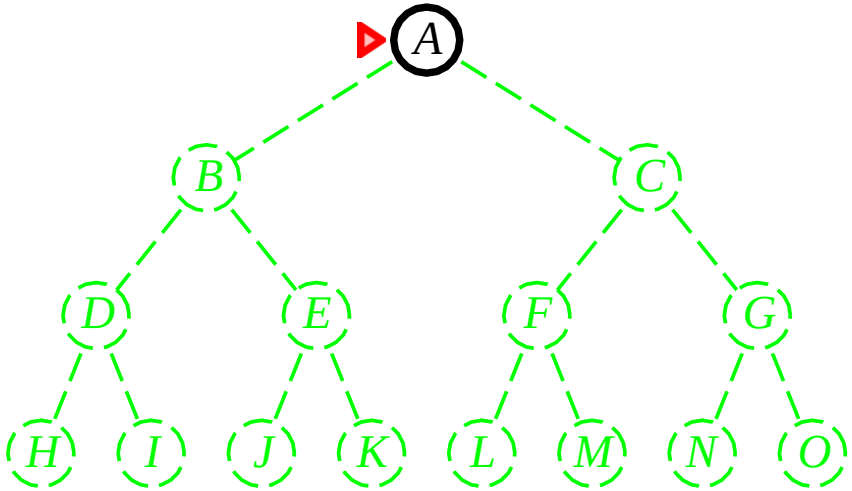
UCS guarantees optimality, but at huge computational cost.

Depth-first search

Expand deepest unexpanded node

Implementation:

fringe = LIFO queue, i.e., put successors at front

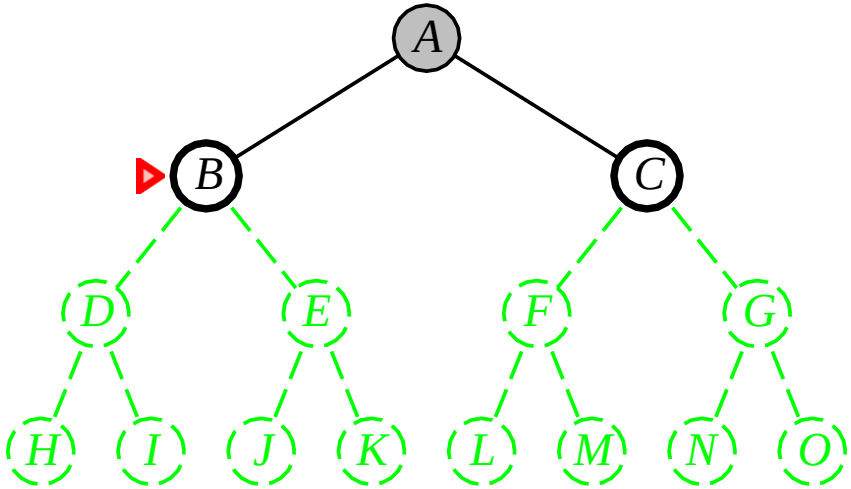


Depth-first search

Expand deepest unexpanded node

Implementation:

fringe = LIFO queue, i.e., put successors at front

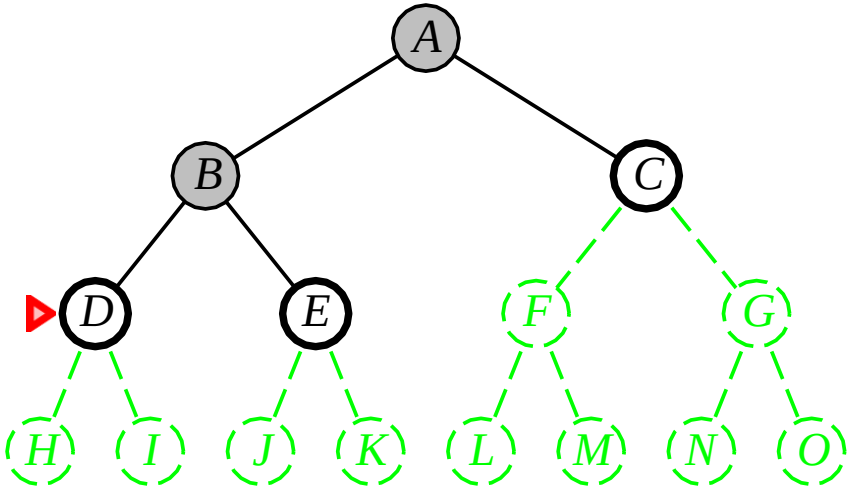


Depth-first search

Expand deepest unexpanded node

Implementation:

fringe = LIFO queue, i.e., put successors at front

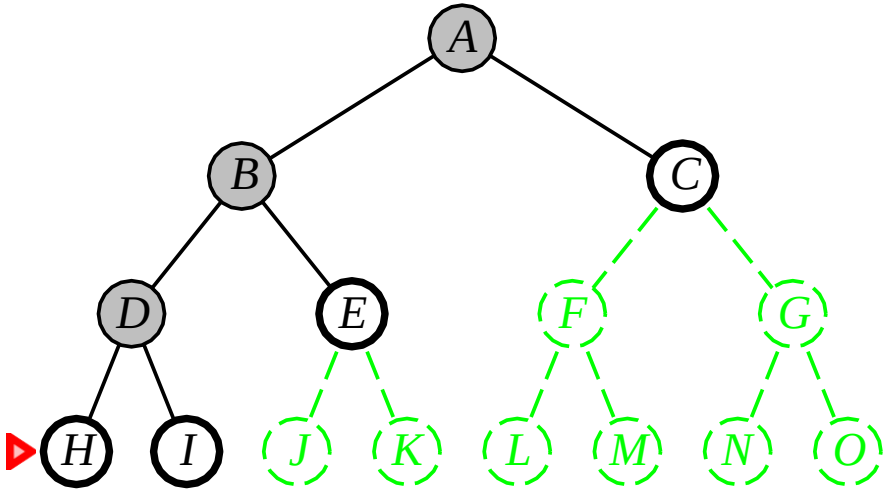


Depth-first search

Expand deepest unexpanded node

Implementation:

fringe = LIFO queue, i.e., put successors at front

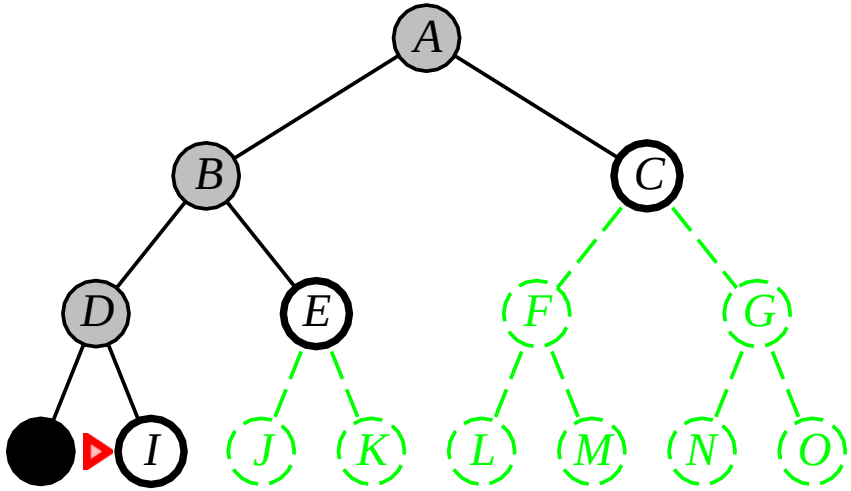


Depth-first search

Expand deepest unexpanded node

Implementation:

fringe = LIFO queue, i.e., put successors at front

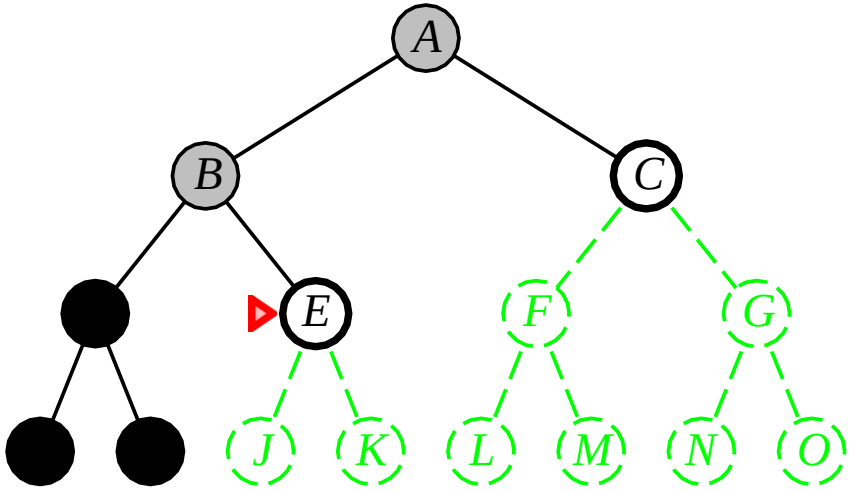


Depth-first search

Expand deepest unexpanded node

Implementation:

fringe = LIFO queue, i.e., put successors at front

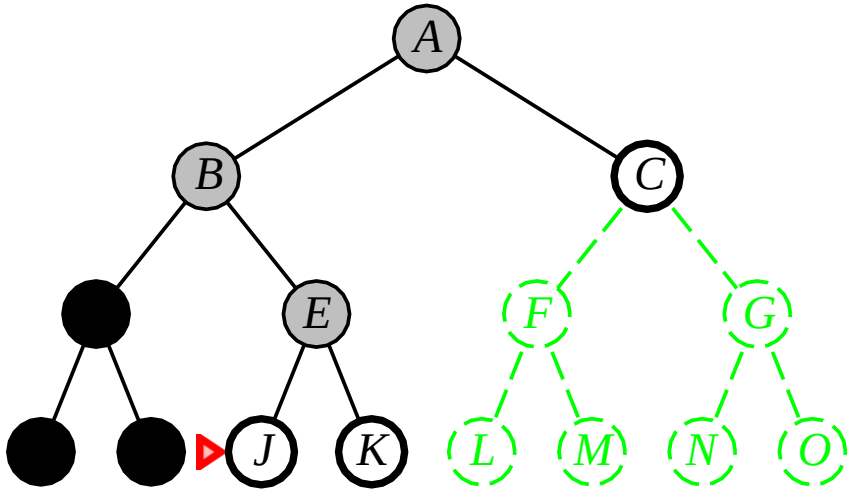


Depth-first search

Expand deepest unexpanded node

Implementation:

fringe = LIFO queue, i.e., put successors at front

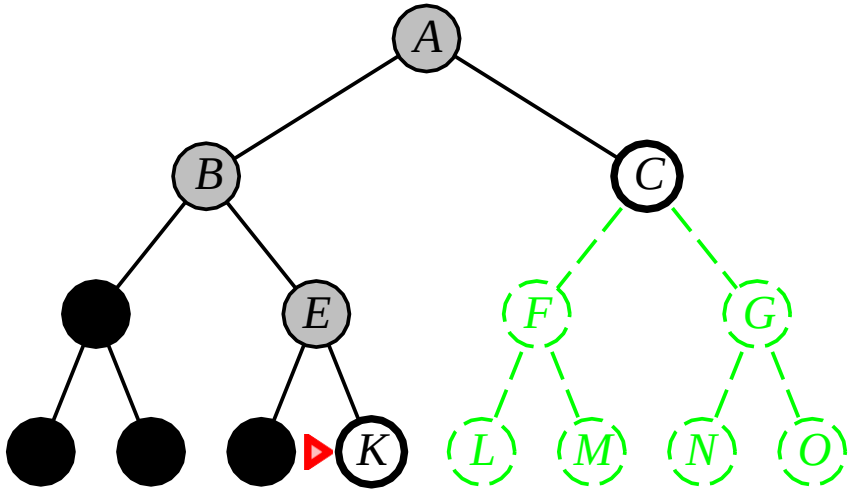


Depth-first search

Expand deepest unexpanded node

Implementation:

fringe = LIFO queue, i.e., put successors at front

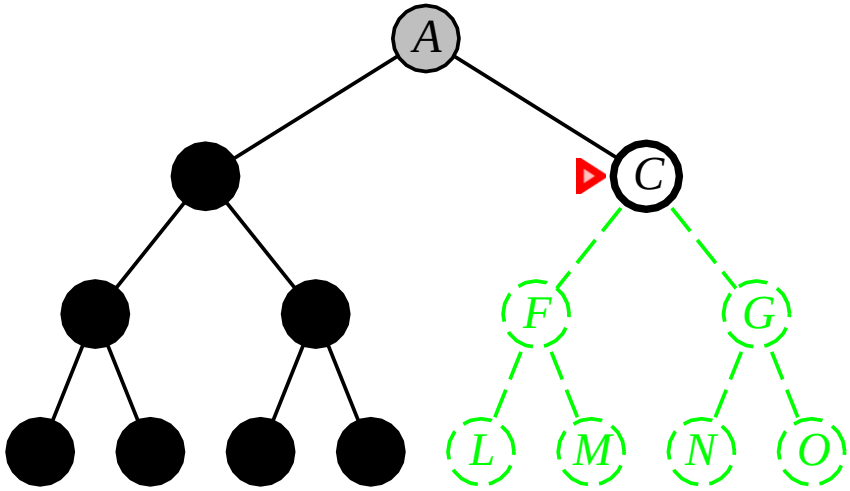


Depth-first search

Expand deepest unexpanded node

Implementation:

fringe = LIFO queue, i.e., put successors at front

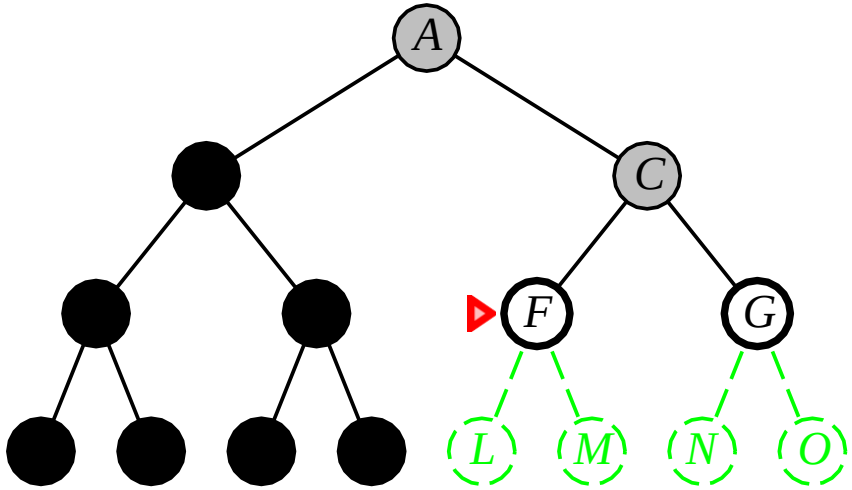


Depth-first search

Expand deepest unexpanded node

Implementation:

fringe = LIFO queue, i.e., put successors at front

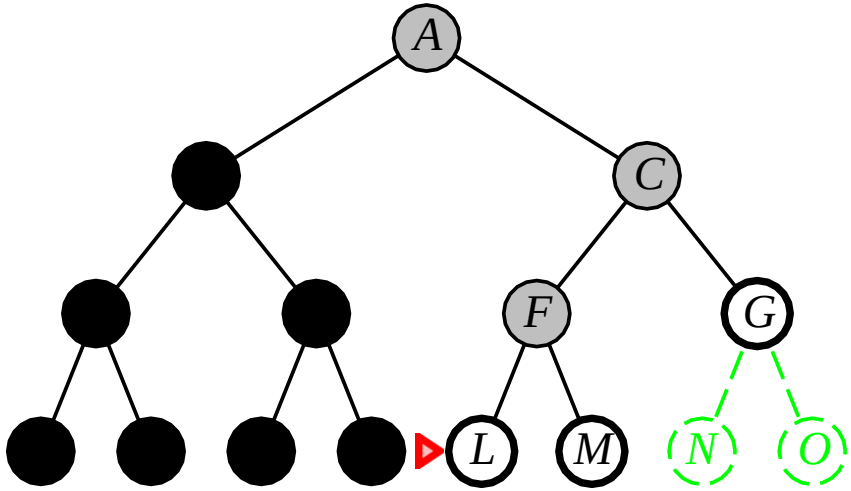


Depth-first search

Expand deepest unexpanded node

Implementation:

fringe = LIFO queue, i.e., put successors at front

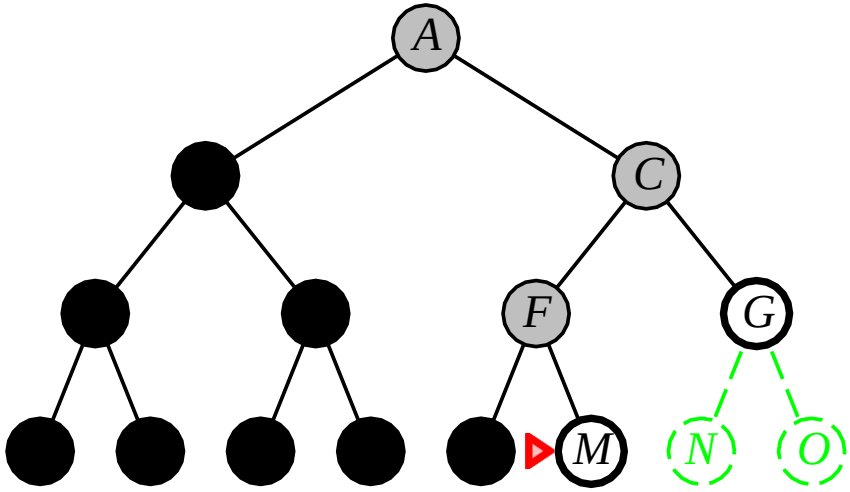


Depth-first search

Expand deepest unexpanded node

Implementation:

fringe = LIFO queue, i.e., put successors at front



Properties of depth-first search

Complete??

Properties of depth-first search

Complete?? No: fails in infinite-depth spaces, spaces with loops

Modify to avoid repeated states along path

⇒ complete in finite spaces

Time??

Properties of depth-first search

Complete?? No: fails in infinite-depth spaces, spaces with loops

Modify to avoid repeated states along path

⇒ complete in finite spaces

Time?? $O(b^m)$: terrible if m is much larger than d

but if solutions are dense, may be much faster than breadth-first

Space??

Properties of depth-first search

Complete?? No: fails in infinite-depth spaces, spaces with loops

Modify to avoid repeated states along path

⇒ complete in finite spaces

Time?? $O(b^m)$: terrible if m is much larger than d

but if solutions are dense, may be much faster than breadth-first

Space?? $O(bm)$, i.e., linear space!

Optimal??

Properties of depth-first search

Complete?? No: fails in infinite-depth spaces, spaces with loops
Modify to avoid repeated states along path
⇒ complete in finite spaces

Time?? $O(b^m)$: terrible if m is much larger than d
but if solutions are dense, may be much faster than breadth-first

Space?? $O(bm)$, i.e., linear space!

Optimal?? No

Time Complexity:

O(b^m)

Space Complexity:

O(bm)

DFS saves memory but may never finish.

Depth-limited search

= depth-first search with depth limit *l*,
returns *cutoff* if any path is cut off by depth limit

Recursive implementation:

```
function DEPTH-LIMITED-SEARCH( problem, limit) returns soln/fail/cutoff
    RECURSIVE-DLS(MAKE-NODE(INITIAL-STATE[problem]), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns soln/fail/cutoff
    cutoff-occurred? ← false
    if GOAL-TEST(problem, STATE[node]) then return node
    else if DEPTH[node] = limit then return cutoff
    else for each successor in EXPAND(node, problem) do
        result ← RECURSIVE-DLS(successor, problem, limit)
        if result = cutoff then cutoff-occurred? ← true
        else if result /= failure then return result
    if cutoff-occurred? then return cutoff else return failure
```

Depth-limited search

Time:
O(b^L)
Space:
O(bL)

We trade completeness for memory safety

Iterative deepening search

```
function Iterative-Deepening-Search( problem) returns a solution
    inputs: problem, a problem

    for depth ← 0 to ∞ do
        result ← Depth-Limited-Search( problem, depth)
        if result /= cutoff then return result
    end
```

Try shallow search → if not found → increase depth.

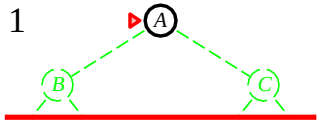
Thus;
IDS combines BFS intelligence + DFS efficiency.

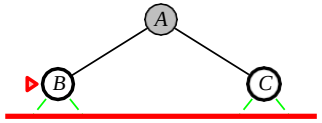
Iterative deepening search *l* = 0

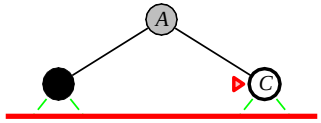
Limit = 0

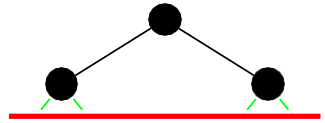
Iterative deepening search *l* = 1

Limit = 1



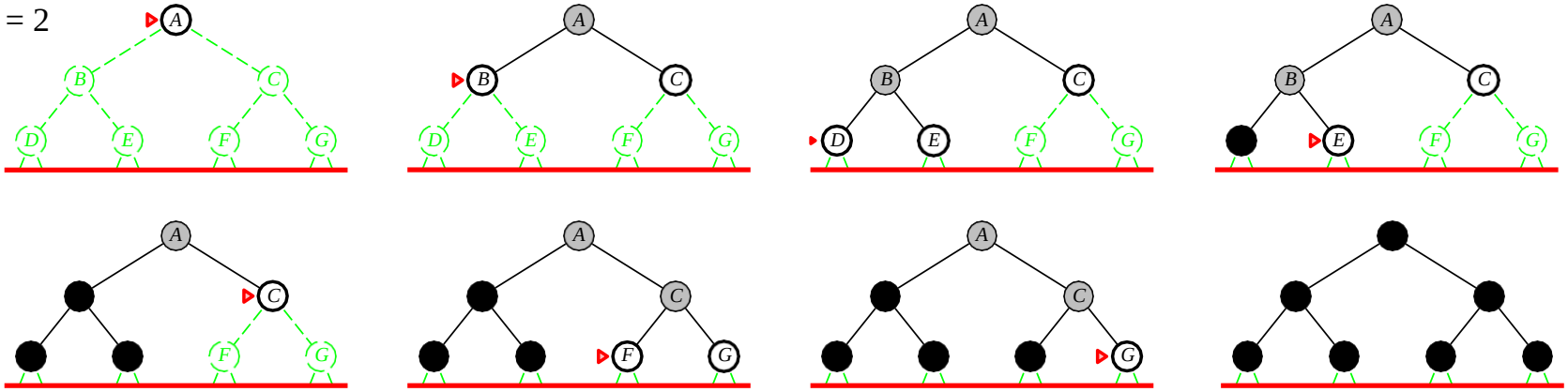






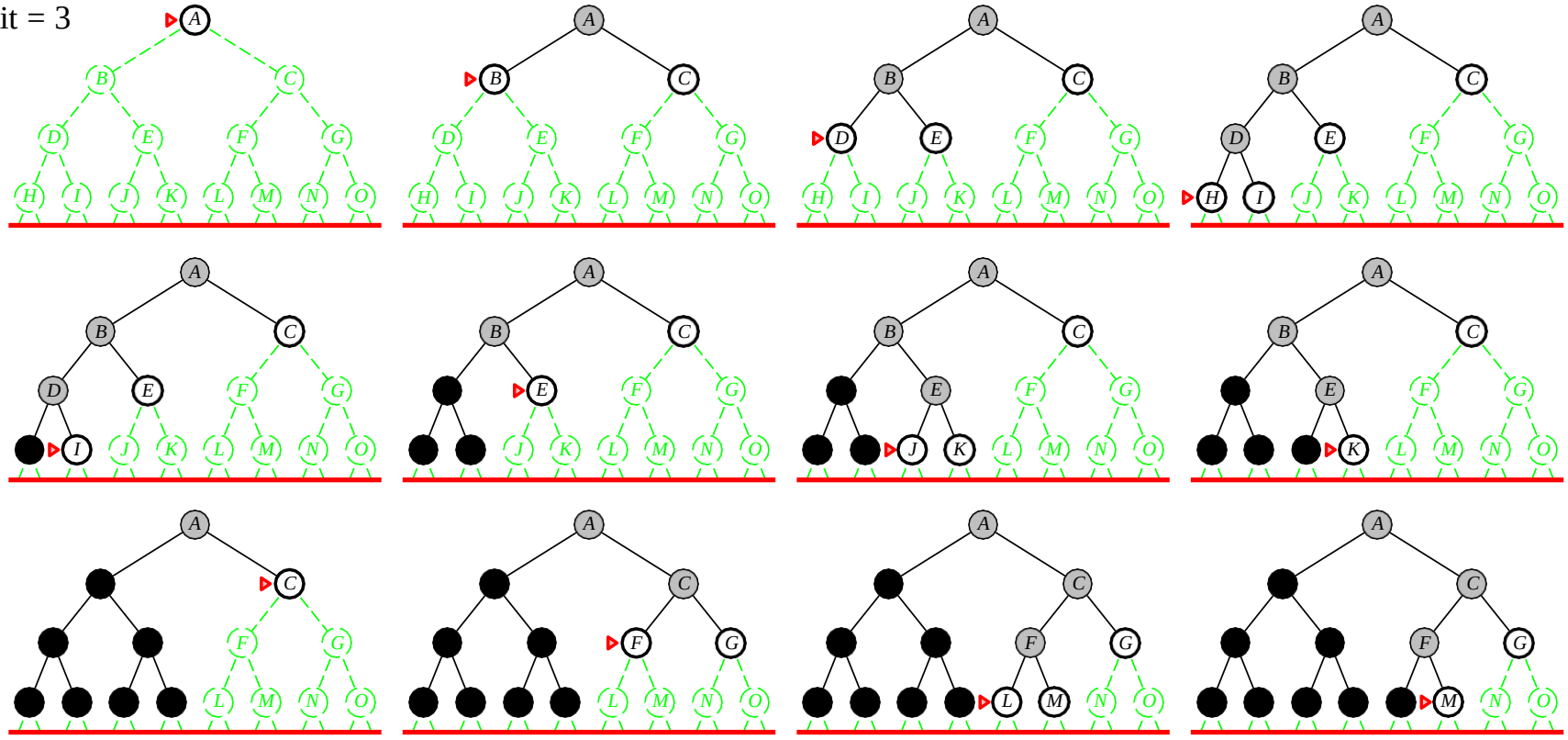
Iterative deepening search *l* = 2

Limit = 2



Iterative deepening search *l* = 3

Limit = 3



Properties of iterative deepening search

Complete??

Properties of iterative deepening search

Complete?? Yes

Time??

Properties of iterative deepening search

Complete?? Yes

Time?? $(d+1)b^0 + db^1 + (d-1)b^2 + \ldots + b^d = O(b^d)$

Space??

Properties of iterative deepening search

Complete?? Yes

Time?? $(d+1)b^0 + db^1 + (d-1)b^2 + \ldots + b^d = O(b^d)$

Space?? $O(bd)$

Optimal??

Properties of iterative deepening search

Complete?? Yes

Time?? $(d+1)b^0 + db^1 + (d-1)b^2 + \ldots + b^d = O(b^d)$

Space?? $O(bd)$

Optimal?? No, unless step costs are constant
Can be modified to explore uniform-cost tree

Numerical comparison for $b = 10$ and $d = 5$, solution at far right leaf:

$N(\text{IDS}) = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450$
$N(\text{BFS}) = 10 + 100 + 1,000 + 10,000 + 100,000 + 999,990 = 1,111,100$

IDS does better because other nodes at depth d are not expanded

BFS can be modified to apply goal test when a node is generated

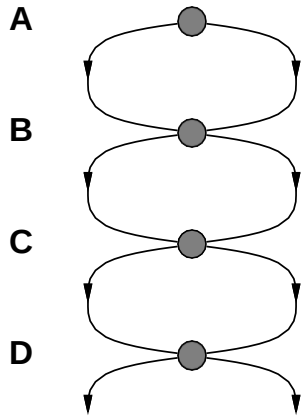
Summary of algorithms

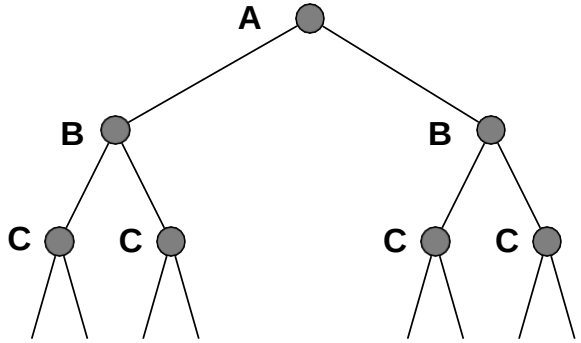
Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes*	Yes*	No	Yes, if $l \geq d$	Yes
Time	b^{d+1}	$b^{\lfloor C^*/\epsilon \rfloor}$	b^m	b^l	b^d
Space	b^{d+1}	$b^{\lfloor C^*/\epsilon \rfloor}$	bm	bl	bd
Optimal?	No*	Yes	No	No	No*

Algorithm	AI Use
BFS	Shortest path
UCS	Cheapest planning
DFS	Memory-limited tasks
IDS	Robotics planning
UCS	Navigation systems

Repeated states

Failure to detect repeated states can cause exponentially more work!





Graph search

```
function GRAPH-SEARCH( problem, fringe) returns a solution, or failure
    closed ← an empty set
    fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
    loop do
        if fringe is empty then return failure
        node ← REMOVE-FRONT(fringe)
        if GOAL-TEST(problem, STATE[node]) then return node
        if STATE[node] is not in closed then
            add STATE[node] to closed
            fringe ← INSERTALL(EXPAND(node, problem), fringe)
    end
```

If we use Tree Search in real-world problems, will it always work?

Draw :

A → B → C → A

Graph search

Tree Search = ignores repeated states
Graph Search = remembers visited states

Human analogy:
You are exploring a city:

- Without memory → you may walk in circles
- With memory → you mark visited places

  ➞ That memory is called Explored Set / Closed List

Graph Search is tree search + memory of visited states to avoid repetition and loops.

Graph search

TREE SEARCH vs GRAPH SEARCH

Feature	Tree Search	Graph Search
Memory	No visited tracking	Uses explored set
Repeated states	Yes	No
Loops	Possible	Prevented
Space	Lower	Higher
Time	Higher	Lower (no repetition)

Graph search trades memory to save time — and that's a smart trade

Graph search

- **Tree search is theoretical; graph search is practical.**
- **Memory saves time.**
- **Loops destroy AI — graph search prevents them**

```
function GRAPH-SEARCH( problem, fringe) returns a solution, or failure
    closed ← an empty set
    fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
    loop do
        if fringe is empty then return failure
        node ← REMOVE-FRONT(fringe)
        if GOAL-TEST(problem, STATE[node]) then return node
        if STATE[node] is not in closed then
            add STATE[node] to closed
            fringe ← INSERTALL(EXPAND(node, problem), fringe)
    end
```


Use hash table for *closed* — constant-time lookup!

Graph search

```
function GRAPH-SEARCH( problem, fringe) returns a solution, or failure

    closed ← an empty set
    fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
    loop do
        if fringe is empty then return failure
        node ← REMOVE-FRONT(fringe)
        if GOAL-TEST(problem, STATE[node]) then return node
        if STATE[node] is not in closed then
            add STATE[node] to closed
            fringe ← INSERTALL(EXPAND(node, problem), fringe)
    end
```


Use hash table for *closed* — constant-time lookup!


Makes all algorithms complete in finite spaces!!

Graph search

```
function GRAPH-SEARCH( problem, fringe) returns a solution, or failure
    closed ← an empty set
    fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
    loop do
        if fringe is empty then return failure
        node ← REMOVE-FRONT(fringe)
        if GOAL-TEST(problem, STATE[node]) then return node
        if STATE[node] is not in closed then
            add STATE[node] to closed
            fringe ← INSERTALL(EXPAND(node, problem), fringe)
    end
```


Use hash table for *closed* — constant-time lookup!


Makes all algorithms complete in finite spaces!!


Makes all algorithms worst-case exponential space!!!

Graph search

```
function GRAPH-SEARCH( problem, fringe) returns a solution, or failure
    closed ← an empty set
    fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
    loop do
        if fringe is empty then return failure
        node ← REMOVE-FRONT(fringe)
        if GOAL-TEST(problem, STATE[node]) then return node
        if STATE[node] is not in closed then
            add STATE[node] to closed
            fringe ← INSERTALL(EXPAND(node, problem), fringe)
    end
```


Use hash table for *closed* — constant-time lookup!


Makes all algorithms complete in finite spaces!!


Makes all algorithms worst-case exponential space!!!


But size of graph often much less than $O(b^d)$!!!!

Graph search makes AI practical, but at the cost of memory

Summary

Problem formulation usually requires abstracting away real-world details to define a state space that can feasibly be explored

Variety of uninformed search strategies

Iterative deepening search uses only linear space
and not much more time than other uninformed algorithms

Graph search can be exponentially more efficient than tree search